

Graph-as-Code

Origem, método e validações

PPGCC - FACOM/UFMS

Luiz F. P. Quirino

2 de setembro de 2026

Esta apresentação é baseada no artigo

“Actions Speak Louder than Prompts”

Finkelshtein, Cucerzan, Jauhar & White

ICLR 2026

Os exemplos, dados e resultados apresentados aqui seguem o texto original. A narrativa foi reorganizada para fins didáticos.

LLMs se tornaram solucionadores de propósito geral.

Com escala, modelos de linguagem adquirem capacidades que ninguém programou: raciocínio em cadeia, geração de código, uso de ferramentas. Funciona muito bem em texto puro.

Grafos textuais são a fronteira natural.

Redes sociais, citações, e-commerce — entidades conectadas onde cada nó carrega texto. Se o LLM entende texto e o grafo codifica relações, combinar os dois promete classificação sem treino.

O desafio: classificar com poucos rótulos.

Rotular nós em grafos é caro. O cenário real é ter 5–10% de labels e precisar classificar o restante. LLMs raciocinam zero-shot — podem resolver isso?

O problema

1 Motivação

Síntese da pergunta central (Finkelshtein et al., ICLR 2026)

“Qual a melhor forma de um LLM interagir com dados de grafos?”

O artigo testa 3 paradigmas de interação e apresenta que **deixar o LLM explorar o grafo gerando código** supera os demais.

O ponto de partida

2 Genealogia

LLMs são bons em linguagem natural, mas ruins em:

- Cálculos (erram contas simples)
- Raciocínio em múltiplos passos (degradam com a profundidade)
- Dados estruturados (grafos, tabelas)

Exemplo

“quantos vizinhos do nó 133 são classe 5?”

Sabe-se que o LLM pode contar errado. A solução surgiu em etapas. Cada uma resolve a limitação da anterior.

2022: Chain-of-Thought – CoT (Wei et al., NeurIPS)

2 Genealogia

Insight: se o LLM explicita o raciocínio passo a passo, erra menos.

Sem CoT:

```
1 User:      Classe do no 133?  
2 LLM:      Classe 3.                                (errado)
```

Com CoT:

```
1 User:      Classe do no 133?  
2 LLM:      0 titulo menciona Reinforcement Learning.  
3           4 dos 5 vizinhos pertencem a classe 5.  
4           Logo, classe 5.                            (correto)
```

Limitação: aritmética por inferência falha em problemas complexos.

2023: PAL – Program-Aided Language Models (Gao et al., ICML)

2 Genealogia

Insight: em vez de raciocinar sobre números, gerar código que calcula.

```
1 neighbors_labels = [5, 5, 5, 5, 1]
2 most_common = Counter(neighbors_labels).most_common(1)[0][0]
```

O LLM traduz o problema em programa. Python executa e retorna 5.

Limitação: o LLM precisa receber *todos os dados* no prompt para gerar o código. Grafo grande estoura a janela de contexto.

2023: ReAct – Reasoning + Acting (Yao et al., ICLR)

2 Genealogia

Insight: buscar dados sob demanda, agindo iterativamente.

```
1 Thought:  Preciso dos vizinhos do no 133.
2 Action:    get_neighbors(133)  ->  [134, 135, 707, 2048]
3
4 Thought:   Preciso do label de cada vizinho.
5 Action:    get_label(134)      ->  5
6 Action:    get_label(135)      ->  5
7 Action:    get_label(707)      ->  5
8 Action:    get_label(2048)     ->  1
9
10 Thought:  Maioria classe 5.   Answer: 5
```

Limitação: 6 chamadas atômicas para uma contagem simples. Lento e *relativamente* caro.

2024: CodeGraph (Cai et al., arXiv)

2 Genealogia

Insight: e se o LLM gerasse código para resolver problemas sobre grafos?

```
1 def count_edges(adj_list):  
2     return sum(len(v) for v in adj_list.values()) // 2  
3  
4 count_edges(graph)
```

O LLM aprende com exemplos (few-shot), gera programas e executa via interpretador. Evita erros aritméticos e torna o raciocínio auditável.

Resultados: melhoria de 1.3% a 58.6% em tarefas de raciocínio sobre grafos (GraphQA), dependendo da tarefa.

Limitação: prova de conceito. Poucos datasets, sem comparação com outros paradigmas (Prompting, Tool-use), sem validação em escala.

Genealogia ~~Completa~~ Lógica

2 Genealogia

- 2022 • **Chain-of-Thought**: pensa passo a passo
| *Limitação: cálculos de cabeça erram*
- 2023 • **PAL**: delega cálculos para código
| *Limitação: precisa de todos os dados antecipadamente*
- 2023 • **ReAct**: busca dados iterativamente
| *Limitação: ações atômicas, sem composição*
- 2024 • **CodeGraph**: código + grafos (prova de conceito)
| *Limitação: poucos datasets, sem validação*
- 2026 • **Graph-as-Code**: síntese validada em escala
14 datasets · múltiplos modelos · node classification

Cada passo resolve a limitação do anterior. Graph-as-Code (GaC) é a convergência natural.

2026: Graph-as-Code (Finkelshtein et al., ICLR)

2 Genealogia

Síntese: código (PAL) + iteração (ReAct) + grafos.

```
1 df.loc[133, 'features']  
2 df.loc[df.loc[133, 'neighbors'], 'label'].value_counts()
```

Passo 1: consulta o texto do nó alvo.

Passo 2: conta os labels dos vizinhos numa expressão só.

Passo 3: LLM combina os sinais e decide.

2 expressões fazem o que ReAct levou 6 chamadas.

A diferença com CodeGraph: GaC é **iterativo com feedback**. Cada expressão é informada pelo resultado da anterior. CodeGraph gera programa inteiro de uma vez sem ver resultados intermediários.

Incorpora e estende a ideia do CodeGraph (Cai et al., 2024).

A ideia central

3 Como funciona

O grafo inteiro vira uma tabela. (DataFrame) Cada linha é um nó:

id	features (o texto)	label (a classe)	neighbors (conexões)
133	"A Reinforcement Learning..."	?	[134, 135, 707, 2048]
134	"Q-Learning for Robot..."	5	[133, 135]
135	"Policy Gradient Methods..."	5	[133, 134, 707]
707	"Temporal Difference..."	5	[133, 135]
2048	"Image Segmentation..."	1	[133]

Um nó: um artigo, 3 atributos: features, neighbors, label.

Uma aresta: citação entre artigos.

Labels visíveis: poucos revelados como referência. O 133 é o alvo.

O mecanismo: perguntar, receber, decidir

3 Como funciona

O LLM funciona como um investigador com acesso a um terminal:

LLM pergunta → Python responde → LLM pergunta → ... →
decide

1. LLM recebe: “classifique o nó 133” + estrutura da tabela
2. Gera uma pergunta em forma de código (expressão pandas)
3. Python executa e devolve a resposta como texto
4. LLM lê a resposta, decide se precisa de mais informação
5. Quando confiante: emite a classe final

Por que funciona: o LLM escolhe o *que* investigar (criatividade).

Python garante que a resposta é exata (execução). Código gerado = explicação auditável.

Exemplo: classificando o nó 133, passo a passo

3 Como funciona

Step 1 — “Sobre o que é este nó?”

```
1 df.loc[133, 'features']
```

→ "A Reinforcement Learning Approach to Job-Shop Scheduling"

O LLM agora sabe: o texto fala de Reinforcement Learning.

Step 2 — “O que os vizinhos dizem?”

```
1 df.loc[df.loc[133, 'neighbors'], 'label'].value_counts()
```

→ {5: 4, 1: 1} (4 vizinhos são classe 5, 1 é classe 1)

O LLM agora sabe: a vizinhança confirma — maioria é classe 5.

Exemplo: classificando o nó 133, passo a passo

Step 3 — decidir

O LLM combina os dois sinais:

1. **Texto** sugere Reinforcement Learning → hipótese: classe 5
2. **Vizinhança** confirma: 4 de 5 vizinhos são classe 5
3. Sinais convergem → confiança alta

Answer: 5 (correto!)

Por que isso é poderoso:

- 2 perguntas bastaram, não precisou ver o grafo inteiro
- Texto e estrutura se complementam (um confirma o outro)
- Se o texto fosse ambíguo, a vizinhança desempataria
- Se vizinhos não tivessem label, o texto sustentaria sozinho

Adaptação flexível: usa o sinal mais informativo disponível.

Validação: escopo do estudo original

4 Validação

Finkelshtein et al. testaram em:

- **14 datasets:** Cora, PubMed, ArXiv, Wiki-CS, Reddit, Cornell, Texas...
- **Múltiplos modelos:** o4-mini, GPT-5, DeepSeek R1, Phi-4, Qwen, Llama...
- **Foco:** node classification (+ shortest path como validação complementar)

Baselines comparados:

Categoria	Métodos
GNNs supervisionadas	GCN, GAT, GraphSAGE (SOTA – State of the Art)
Semi-supervisionado	Label Propagation
LLM zero-shot	Prompting, Tool-use, Graph-as-Code

Resultados: eficiência de tokens

4 Validação

Consumo de tokens por nó (datasets de texto longo, Tabela 6 do paper):

Dataset	Prompting 2-hop	GraphTool	GaC
Reddit	42.100	16.100	14.900
Photo	104.700	43.300	41.500
Wiki-CS	1.157.000 (estoura)	61.700	57.100

Achado principal: GaC usa $20\times$ menos tokens que Prompting 2-hop em Wiki-CS. Em grafos densos, Prompting simplesmente não cabe no contexto.

Nota: os resultados de accuracy variam por dataset e modelo. O paper reporta resultados específicos por combinação, não faixas gerais. Consultar Tabelas 2–5 do artigo para números exatos.

Ablação e achados principais

4 Validação

Finkelshtein remove componentes para medir a contribuição de cada sinal:

- Truncar texto: Prompting piora. **GaC compensa com estrutura.**
- Remover arestas: Prompting piora. **GaC compensa com texto.**
- Esconder labels: ambos pioram, mas GaC degrada menos.

Ablação e achados principais

4 Validação

Tradução direta da introdução do artigo (Finkelshtein et al., ICLR 2026):

***Achados.** Nossa avaliação produz percepções e diretrizes-chave para a aplicação de LLMs a dados de grafo:*

- *Graph-as-Code alcança o melhor desempenho geral, com ganhos especialmente grandes em **grafos de texto longo ou de alto grau**, onde o prompting rapidamente esgota o orçamento de tokens.*
- *Todos os modos de interação LLM-grafo são eficazes em **grafos heterofílicos**, desafiando a suposição comum de que eles colapsam sob baixa homofilia.*
- *Graph-as-Code é capaz de **deslocar flexivelmente** sua dependência entre estrutura, features ou rótulos, aproveitando o tipo de entrada mais informativo.*

Por que Graph-as-Code funciona

4 Validação

Três razões técnicas:

1. **Eficiência de tokens** — consulta sob demanda, não serializa tudo
57k tokens (GaC) vs. 1.157k (Prompting) no Wiki-CS
2. **Composicionalidade** — operações complexas em 1 expressão
`.value_counts()` = buscar + filtrar + contar
3. **Precisão algorítmica** — código executa, não “pensa”
MSE (Mean Squared Error) = 0.0 em shortest path (próximo slide)

Adaptação flexível (Achado 9): quando o texto é enganoso, GaC desloca para sinais estruturais sem reconfiguração.

Precisão algorítmica: shortest path

4 Validação

Tarefa: distância mínima entre dois nós.

Prompting/Tool-use: LLM conta saltos de cabeça e erra em grafos grandes.

Graph-as-Code: LLM gera código que resolve exatamente:

```
1 G = nx.from_pandas_edgelist(  
2     df.explode('neighbors'), source='id', target='neighbors')  
3 nx.shortest_path_length(G, source=42, target=133)
```

Resultado: MSE = 0.0 (erro zero) para GaC.

Princípio: LLM decide *o que* calcular (criatividade).

Python garante que o cálculo é correto (execução).

GaC vs. GNNs (Graph Neural Networks) supervisionadas

5 GaC vs GNNs

	GNNs (GCN, GAT...)	Graph-as-Code
Treino	GPU + milhares de exemplos	Zero-shot (LLM congelado)
Accuracy	Geralmente maior (SOTA)	Menor (trade-off por flexibilidade)
Poucos rótulos	Sofre	Funciona (10% basta)
Explicabilidade	Caixa-preta	Código = explicação
Novo domínio	Retreinar	Funciona direto
Latência	ms	s-min por nó

GaC não substitui GNNs. É alternativa para cenários com poucos rótulos, poucos recursos, ou necessidade de explicabilidade.

Limitações e fronteiras

6 Limitações

GaC não tem vantagem quando:

- Grafo esparso (vizinhança cabe no prompt)
- Nós isolados (sem vizinhança para explorar)
- Latência importa (40-160s por nó)
- Todos os labels disponíveis (GNNs dominam)

Limitações e fronteiras

6 Limitações

GaC é a melhor opção quando:

- Grafo denso + poucos rótulos + texto longo
- Explicabilidade necessária (código auditável)
- Novo domínio/idioma sem dados de treino
- Custo importa

Em vez de descrever o grafo inteiro para o LLM, deixamos ele explorar por conta própria, gerando código.

1. Grafo representado como DataFrame pandas
2. LLM gera expressões para explorar vizinhança sob demanda
3. Executor seguro avalia e devolve resultado
4. LLM itera até decidir

Funciona onde Prompting estoura (grafos densos) e Tool-use é lento (ações atômicas).

Referências

7 Conclusão

- Finkelshtein et al. (2026). “Actions Speak Louder than Prompts”. ICLR.
- Gao et al. (2023). “PAL: Program-Aided Language Models”. ICML.
- Yao et al. (2023). “ReAct: Synergizing Reasoning and Acting in LLMs”. ICLR.
- Cai et al. (2024). “CodeGraph: Enhancing Graph Reasoning of LLMs with Code”. arXiv:2408.13863.
- Wei et al. (2022). “Chain-of-Thought Prompting Elicits Reasoning in LLMs”. NeurIPS.
- Wei et al. (2022). “Emergent Abilities of Large Language Models”. TMLR.

Graph-as-Code

Obrigado!